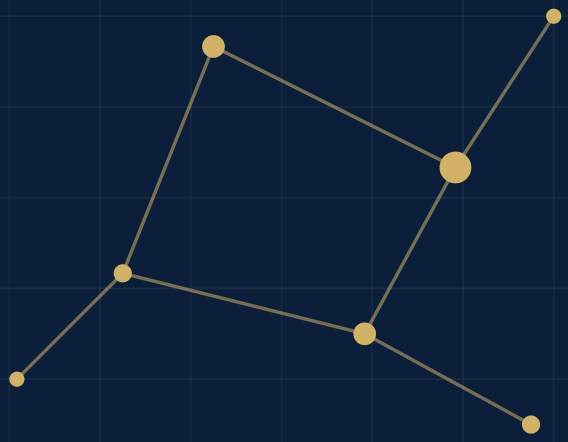


03 — DAY 2 · HACKATHON

Hackathon Guide

Problem statements, rules and judging

Eight industry problem statements, the common tech stack, the agent rule, team roles, the day plan, the judging rubric and the submission checklist.

TEAMS	TIME	PROBLEMS
40 teams	10:00 to 4:00	8 industry cases

PREPARED BY

Sathish Kumar K.

AI Educator

simplilearn

Student reference · Guide 3 of 3

November 2026

Contents

Problem statements, rules and judging. Follow the sections in order; each one ends with a checkpoint or checklist.

01 Hackathon at a glance 3

02 Important links 3

03 Tech stack (same for all teams) 3

04 The agent rule (must pass) 4

05 Team roles 5

06 Setup steps (first 45 minutes) 5

07 Problem statements 7

08 Day 2 timeline and checkpoints 15

09 Rules 16

10 Judging rubric (100 marks) 16

11 Submission checklist 17

12 Troubleshooting and help 19

PREPARED BY

Sathish Kumar K., AI Educator

PROGRAMME

SRM University workshop

EDITION

November 2026

01 Hackathon at a glance

In one day, your team builds a working, industry-style web app with a real AI agent, deploys it on Vercel and presents it to judges.

Item	Detail
Teams	40 teams
Problem statements	8 industry problems, 5 teams per problem
Duration	10:00 to 4:00, with lunch 12:30 to 1:30
Base code	One common starter template for all teams
Final output	Working app + GitHub repo + Vercel link + 5 slides

Everything you learned on Day 1 (Next.js, Supabase, Gemini and function calling) is what you use today. You are not expected to build from zero. The starter template gives you the setup, so you can spend your time on the problem.

02 Important links

Keep this table open all day.

What	Link	Use it for
Starter template (GitHub)	github.com/learn2simple/srm-agent-hackathon-starter	Click **Use this template**. It is the base for every team.
Seed data folder	.../srm-agent-hackathon-starter/tree/main/seed	SQL file for your problem statement
Supabase	supabase.com	Create your team database
Google AI Studio	aistudio.google.com	Create your team Gemini API key
Vercel	vercel.com	Deploy your app (log in with GitHub)
Submission form	Google Form, shared by the trainer on the day	Submit your 5 slides, repo link and Vercel link between 3:30 and 4:00

03 Tech stack (same for all teams)

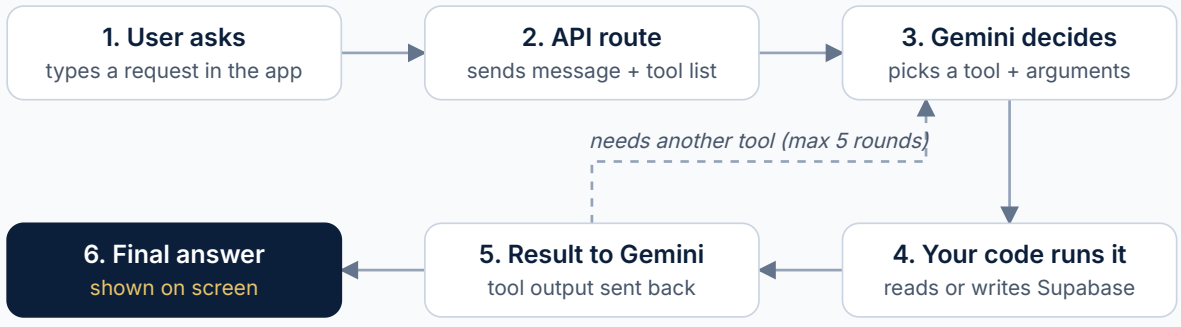
All 40 teams use the same stack. This keeps help and judging fair. Do not switch to another framework, database or LLM.

Layer	Technology	Required
Frontend	React + Next.js (App Router) + Tailwind CSS	Mandatory
Language	TypeScript	Mandatory
Backend	Next.js API Route Handlers	Mandatory
Validation	Zod	Mandatory
Database	Supabase PostgreSQL + Supabase JS SDK	Mandatory
LLM	Google Gemini API (`@google/genai`)	Mandatory
Agent	Gemini Function Calling	Mandatory
Version control	Git + GitHub (one team repo)	Mandatory
Deployment	Vercel	Mandatory
API testing	Postman or REST Client	Recommended
AI coding tools	GitHub Copilot, Claude Code, OpenAI Codex	Allowed

All services above have a free plan. You do not need to pay for anything.

04 The agent rule (must pass)

Your app must have at least one real function-calling workflow. A chatbot that only replies with text does not count, and the team will lose all 30 agent marks.



The agent loops between Gemini and your tools until it can answer.

Gemini never touches your database directly. It only asks for a tool. Your code runs the tool and sends the result back.

Your agent passes if:

- It has at least 2 tools declared to Gemini
- At least 1 tool writes to Supabase (insert or update)
- The user asks in plain English and Gemini chooses the tool by itself
- Tool arguments are checked with Zod before the tool runs
- The result is visible on screen (not only in the console)

It does not pass if:

- Gemini only gives a text reply with no tool call
- Your code decides which function to call with `if` or `switch` on the user's words
- Tools return hard-coded data instead of reading Supabase

05 Team roles

Each team has up to 5 members, and each member owns one area. Everyone writes code and everyone must be able to explain the app.

Role	Owns	Main files
Team Lead	Plan, GitHub repo, Vercel deploy, testing, final demo	`README.md`, Vercel settings
Frontend	Pages, components, Tailwind styling	`app/` pages, `components/`
Backend / API	API routes and Zod schemas	`app/api/`, `lib/schemas.ts`
Database	Supabase tables, seed data, queries	`seed/`, `lib/supabase.ts`
AI / Agent	Gemini tools, system prompt, agent loop	`lib/agent/`

Team of 3 or 4? Combine roles: the Team Lead also owns the Database, and Frontend also owns the Backend / API. The AI / Agent role always stays with one person.

Write each member's name and role in your README.

06 Setup steps (first 45 minutes)

Finish these steps before you write any feature code. You did the same setup on Day 1, so it should take about 15 minutes. Deploy to Vercel early, not at 3:30.

1. **Team Lead: create the team repo.** Open the starter template on GitHub, click **Use this template**, and name the repo `team-<number>-ps<number>` (for example `team-07-ps3`). Add the other members as collaborators.
2. **Everyone: clone and install.** In a folder outside OneDrive (for example `C:\Projects`), run `git clone <your team repo link>`, then `cd` into the folder and run `npm install`.
3. **Database member: create Supabase.** Create a new project at supabase.com (region: Mumbai). Open **SQL Editor** and run `seed/00_starter_example.sql` first, then the seed file for your problem (for example

`seed/ps3_clinic_appointments.sql`).

4. **Database member: copy the keys.** In Supabase, go to **Project Settings** and copy the **Project URL** (only up to `.supabase.co`) and the **secret key** (also called the service_role key). Both must come from the same project.
5. **AI member: create the Gemini key.** In Google AI Studio, click **Get API key** and create one key for the team. Do not use one key for many teams.
6. **Everyone: create `.env.local`.** Copy `.env.example` to `.env.local` and fill in the values below. Share the keys only in your private team chat.
7. **Everyone: run the app.** Run `npm run dev` and open `http://localhost:3000`. Enter the passcode and try the sample prompt: *"Add a task to call the vendor tomorrow, then show all my tasks."* You should see the agent call 2 tools.
8. **Team Lead: deploy to Vercel.** On vercel.com, log in with GitHub, click **Add New → Project**, and import your team repo. Add the same 5 environment variables, then click **Deploy**. Every push to `main` now redeploys automatically.

Variable	Where to get it
`GEMINI_API_KEY`	Google AI Studio
`GEMINI_MODEL`	`gemini-3.8-flash` (unless the trainer gives another)
`SUPABASE_URL`	Supabase Project Settings
`SUPABASE_SECRET_KEY`	Supabase Project Settings (secret / service_role key)
`APP_PASSCODE`	Your team picks one. It protects your app and your Gemini quota.

Never commit `.env.local` to GitHub, and never put `SUPABASE_SECRET_KEY` in frontend code. The starter template already blocks both. Open `/api/health` any time to check your settings.

07 Problem statements

There are 8 problems, and 5 teams get each one. Your team gets its problem by lottery at kickoff. All company names and records in the seed data are made up.

No.	Problem	Industry	Seed file
PS1	E-commerce Customer Support Agent	Retail	`ps1_ecommerce_support.sql`
PS2	Inventory Restock Agent	Supply chain	`ps2_inventory_restock.sql`
PS3	Clinic Appointment Agent	Healthcare	`ps3_clinic_appointments.sql`
PS4	IT Helpdesk Ticket Agent	IT services	`ps4_it_helpdesk.sql`
PS5	Expense Claim Assistant	Finance / HR	`ps5_expense_claims.sql`
PS6	Job Match Agent	HR tech / Careers	`ps6_job_match.sql`
PS7	Delivery Dispatch Agent	Logistics	`ps7_delivery_dispatch.sql`
PS8	Smart Banking Assistant	Banking / Fintech	`ps8_smart_banking.sql`

The tools listed under each problem are suggestions. You can rename them, merge them or add more, but the agent rule above must still pass.

PS1: E-commerce Customer Support Agent (ShopEase)

Business user: a customer of an online store. **Logged-in user:** customer `C001` (Priya Raman).

Problem: the support team answers the same questions all day: where is my order, can I return this, my order is late. Customers wait hours for simple answers.

Build: a support chat where the customer asks in plain English. The agent checks the order, explains its status, starts a return when the return policy allows it, and raises a ticket for a human when it cannot help.

- **Pages:** Support chat, My Orders, My Returns
- **Tables:** `customers`, `products`, `orders`, `order_items`, `returns`, `support_tickets`
- **Suggested tools:** `getMyOrders`, `getOrderDetails`, `checkReturnEligibility`, `createReturnRequest` (writes), `escalateToHuman` (writes)
- **Business rule:** a return is allowed only for delivered orders, within the product's `return_window_days` from the delivery date.
- **Demo prompts:** "Where is my order 1003?" · "I want to return the shoes from my last order, they are too small." · "My order is 4 days late and nobody is answering."
- **Stretch:** order cancellation for orders not yet shipped, and an admin page for tickets.

PS2: Inventory Restock Agent (StockPilot)

Business user: a store manager at a supermarket.

Problem: items run out of stock because nobody checks stock levels against the reorder level in time. Purchase orders are made by hand in a spreadsheet.

Build: a manager dashboard with an agent. The manager asks what needs restocking. The agent finds low-stock items, groups them by supplier, and creates draft purchase orders with quantities and total cost.

- **Pages:** Dashboard (low-stock count, value), Products, Purchase Orders, Agent chat
- **Tables:** `suppliers`, `products`, `purchase_orders`, `purchase_order_items`
- **Suggested tools:** `getLowStockItems`, `getSupplierDetails`, `estimateDaysOfStockLeft`, `createPurchaseOrder` (writes), `updateStock` (writes)
- **Business rule:** an item is low when `stock_qty <= reorder_level`. Order `reorder_qty` units. One purchase order per supplier.
- **Demo prompts:** "What do I need to reorder this week?" · "Create purchase orders for all low-stock dairy items." · "Which items will run out in the next 3 days?"
- **Stretch:** approve or cancel a purchase order, and receive stock when it arrives.

PS3: Clinic Appointment Agent (CareSlot)

Business user: a patient of a multi-speciality clinic. **Logged-in user:** patient `P001` (Arjun Mehta).

Problem: patients call the front desk just to book, move or cancel appointments. Lines are busy and the desk staff cannot keep up.

Build: a booking assistant. The patient says what they need in plain words. The agent suggests the right department, shows free slots, and books, reschedules or cancels the appointment.

- **Pages:** Booking chat, Doctors, My Appointments
- **Tables:** `departments`, `doctors`, `slots`, `patients`, `appointments`
- **Suggested tools:** `searchDoctors`, `getAvailableSlots`, `bookAppointment` (writes), `rescheduleAppointment` (writes), `cancelAppointment` (writes), `getMyAppointments`
- **Business rule:** a slot can be booked only once. Booking marks the slot as booked; cancelling frees it.
- **Safety rule:** the agent must never diagnose or suggest medicine. It only helps with booking.
- **Demo prompts:** "I have a skin rash, book me with a doctor on Friday morning." · "Move my appointment to the next free slot." · "Which Tamil-speaking doctors are free tomorrow?"
- **Stretch:** a doctor's view of the day's appointments.

PS4: IT Helpdesk Ticket Agent (DeskBot)

Business user: an employee at a software company. **Logged-in user:** employee `E101` (Karthik S).

Problem: the IT team gets flooded with simple tickets like password resets and VPN issues. Most have known fixes, and the rest go to the wrong team.

Build: a helpdesk chat. The agent first searches the knowledge base and gives the fix. If the issue is still not solved, it creates a ticket with the right category, priority and support team.

- **Pages:** Helpdesk chat, My Tickets, IT Admin board
- **Tables:** `employees`, `support_teams`, `kb_articles`, `tickets`, `sla_rules`
- **Suggested tools:** `searchKnowledgeBase`, `createTicket` (writes), `getMyTickets`, `getTicketStatus`, `updateTicket` (writes)
- **Business rule:** use `sla_rules` for priority. P1 = many users blocked or a security issue, P2 = one user fully blocked, P3 = work slowed, P4 = a request.
- **Demo prompts:** "My VPN keeps disconnecting since morning." · "I clicked a strange link in an email." · "What is the status of my laptop ticket?"
- **Stretch:** the admin board lets IT staff change status and the agent can summarise all open P1 tickets.

PS5: Expense Claim Assistant (ClaimEase)

Business user: an employee who spends money on work. **Logged-in user:** employee `E001` (Meera Nair, grade L2).

Problem: employees submit claims that break the policy, and the finance team spends hours checking each one by hand.

Build: a claim assistant. The employee describes the expense in plain words. The agent finds the right category, checks it against the policy for the employee's grade, saves the claim, and flags it if it breaks a rule.

- **Pages:** New claim chat, My Claims, Finance review
- **Tables:** `employees`, `policy_rules`, `claims`
- **Suggested tools:** `getEmployee`, `getPolicyForCategory`, `checkPolicyLimit`, `createClaim` (writes), `getMyClaims`
- **Business rule:** if the amount is above `max_amount` for that grade and category, save the claim with status `flagged` and the reason. Claims older than 60 days are not allowed.
- **Demo prompts:** "I spent 2,400 rupees on a client dinner yesterday, submit it." · "How much can I claim for a hotel in Mumbai?" · "Show my pending claims."
- **Stretch:** the finance review page lets finance staff approve or reject claims.

PS6: Job Match Agent (CareerMatch)

Business user: a final-year student looking for a job. **Logged-in user:** student `S001` (Divya K).

Problem: students apply to jobs that do not match their skills or eligibility, and miss the ones that fit them well.

Build: a job search assistant. The agent reads the student's profile, searches jobs, scores each match, explains the missing skills, and saves the best jobs to a shortlist.

- **Pages:** My Profile, Job search chat, My Shortlist
- **Tables:** `profiles`, `jobs`, `shortlists`
- **Suggested tools:** `getMyProfile`, `searchJobs`, `scoreJobMatch`, `saveToShortlist` (writes), `updateMySkills` (writes)
- **Business rule:** skip jobs where the student's CGPA is below `min_cgpa` or the `apply_by` date has passed.
- **Demo prompts:** "Find jobs in Chennai that match my skills and save the top 3." · "Which skills am I missing for data analyst roles?" · "I just learned Docker, add it to my profile and search again."
- **Stretch:** profile editing page, and a cover note draft for a shortlisted job.

PS7: Delivery Dispatch Agent (RouteWise)

Business user: a dispatcher at a city delivery hub. **Logged-in user:** dispatcher `D01` (Anitha V), Chennai hub.

Problem: the dispatcher assigns deliveries to riders by hand from a spreadsheet. Some riders get overloaded while others wait, riders are sent to the wrong area, and failed deliveries are forgotten instead of being rescheduled.

Build: a dispatch assistant. The dispatcher asks in plain English. The agent finds pending deliveries by zone, picks riders who are on shift in the same zone with free capacity, assigns the deliveries, and reschedules failed ones.

- **Pages:** Dispatch chat, Deliveries board (Pending, Assigned, Out for delivery, Delivered, Failed), Riders with their load
- **Tables:** `zones`, `riders`, `rider_shifts`, `deliveries`, `delivery_attempts`
- **Suggested tools:** `getPendingDeliveries`, `getAvailableRiders`, `getRiderLoad`, `assignDelivery` (writes), `rescheduleDelivery` (writes), `getFailedDeliveries`
- **Business rule:** a rider can only take deliveries in their own zone, on a day they have a shift, and up to their `capacity`. A delivery can have at most 3 attempts. Use the database functions `assign_delivery()` and `reschedule_delivery()`; they check every rule.
- **Demo prompts:** "Assign all pending Velachery deliveries for this evening." · "Which riders still have free capacity today?" · "Reschedule yesterday's failed deliveries to tomorrow morning."
- **Stretch:** a rider view that marks a delivery delivered or failed, and a daily summary of cash-on-delivery amounts per rider.

PS8: Smart Banking Assistant (PayWise)

Business user: a retail bank customer using net banking. **Logged-in user:** customer `B001` (Rohan Iyer).

Problem: customers call the bank or visit the branch for simple things: checking a balance, finding out where their money went, moving money between their own accounts, and setting a spending limit. The call centre is overloaded with these requests.

Build: a banking assistant. The customer asks in plain English. The agent shows balances, explains spends by category, moves money between the customer's own accounts, and sets budget alerts.

- **Pages:** Banking chat, My Accounts (balances), Transactions with category filter, My Budgets with a progress bar
- **Tables:** `customers`, `accounts`, `transfer_limits`, `transactions`, `budgets`
- **Suggested tools:** `getMyAccounts`, `getTransactions`, `getSpendSummary`, `transferFunds` (writes), `setBudgetAlert` (writes), `getBudgetStatus`
- **Business rule:** transfers go only between the customer's own accounts, never from a frozen account, never more than the balance, and within the per-transfer and daily limits for the customer's KYC level. Use the database function `transfer_funds()`; it checks every rule and updates both balances together.
- **Safety rule:** the agent must never ask for or show a PIN, password, OTP or full account number, and must not give investment advice.
- **Demo prompts:** "How much did I spend on food this month, and am I over my budget?" · "Move 20,000 from savings to my salary account." · "Set a 4,000 rupee monthly budget for shopping and alert me at 75%."
- **Stretch:** a monthly spend chart by category, and an alert banner when a budget passes its alert level.

08 Day 2 timeline and checkpoints

The day runs from 10:00 to 4:00, with lunch from 12:30 to 1:30. Guide 7 (Hackathon Prompt Handbook) gives the prompt to use at each step.

Time	Activity
10:00 – 10:15	Kickoff, rules, problem lottery
10:15 – 10:30	Read your problem, choose roles, write `AGENTS.md`
10:30 – 10:45	Quick setup: new repo, your seed data, run (same as Day 1)
10:45 – 11:00	Plan with AI. Show the plan to your mentor
11:00 – 12:30	Build the agent: read tools, first write tool, system prompt
12:30	**Checkpoint:** at least 1 of your own Gemini tools works end to end
12:30 – 1:30	Lunch
1:30 – 2:45	API route, screens, second write tool, extra features

Time	Activity
2:45 – 3:30	Test, fix and README; Team Lead and one member make the 5 slides
3:30	**Code freeze:** 2 or more tools work, 1 writes to Supabase, Vercel link live
3:30 – 4:00	Submit in the Google Form: 5 slides, GitHub link, Vercel link

Each problem statement has its own mentor, who looks after its 5 teams. Your mentor visits your team at each checkpoint. If you miss a checkpoint, tell your mentor at once so they can help you catch up.

After code freeze, pushes to GitHub are not counted. The judges look at the last commit before 3:30.

09 Rules

1. Use the common tech stack. Do not change the framework, database or LLM.
2. Start from the starter template. Code written before the hackathon (other than the template) is not allowed.
3. AI coding tools are allowed. Any team member must be able to explain any part of the code when a judge asks.
4. Every team member must commit to the team repo. Judges check the commit history.
5. Never commit API keys or `.env.local`. A key found in the repo costs 5 marks.
6. Use only the seed data and made-up data. Do not put real personal data in the app.
7. Stay with your own problem statement. You can add features, but the core problem must be solved.
8. Do not copy code from another team. Teams with the same code will both be disqualified.
9. If the Gemini quota runs out, tell your mentor. Do not use another team's key.
10. The judges' decision is final.

10 Judging rubric (100 marks)

Judges review your 5 slides, GitHub repo and Vercel link from the Google Form, and may ask your team to show the app. All judges use this same rubric.

Area	Marks	What the judges check
Agentic workflow	30	2 or more tools, 1 writes to Supabase, Gemini picks tools by itself, Zod checks the arguments, the agent handles a wrong or unclear request
Working app	25	The core problem is solved end to end on the Vercel link, with no crash during the demo
Code and GitHub	15	Clean folders, commits from every member, a clear README, no keys in the repo
UI and UX	10	Easy to use, works on a phone screen, the user can see what the agent did

Area	Marks	What the judges check
Demo and explanation	20	Clear business story, every member can explain their part, answers to questions

Tie-break: the team with the higher agentic workflow score wins.

Tip: start your demo with the business user and the problem in one line. Then show one full request where the agent calls 2 or more tools and the data changes on screen.

11 Submission checklist

The Team Lead submits the Google Form between 3:30 and 4:00. Tick each item off before 3:30.

- [ ] GitHub repo link (public, or shared with the trainer)
- [ ] Live Vercel link, with the passcode written in the form
- [ ] README filled in, using the template below
- [ ] At least 2 Gemini tools working, 1 of them writing to Supabase
- [ ] Commits from every team member
- [ ] No API keys anywhere in the repo
- [ ] 5 slides ready (title and team, problem, solution and demo, how the agent works, results and links), as a PDF or a view-only Google Slides link
- [ ] Backup plan ready: the app also runs on localhost

Copy this into your `README.md` and fill it in.

FILE CONTENT

```
# <App name>: <Problem statement title>

Team <number> | PS<number> | SRM Agentic AI Hackathon

## The problem
<Who is the business user and what is their problem, in 2 lines>

## What our app does
<3 to 5 bullet points>

## Live demo
- Vercel link: <link>
- Passcode: <passcode>
- Try this prompt: "<your best demo prompt>"

## Agent tools
| Tool | What it does | Reads or writes |
| --- | --- | --- |
| <toolName> | <one line> | Reads |
| <toolName> | <one line> | Writes |

## Database tables
<List each table and what it stores>

## Tech stack
Next.js, React, TypeScript, Tailwind CSS, Supabase, Gemini API (function calling), Zod, Vercel

## Team
| Name | Role |
| --- | --- |
| <name> | Team Lead |
| <name> | Frontend |
| <name> | Backend / API |
| <name> | Database |
| <name> | AI / Agent |

## Run it locally
1. npm install
2. Copy .env.example to .env.local and fill in the keys
3. npm run dev
```

12 Troubleshooting and help

Check this table first. If it does not fix the issue in 10 minutes, raise the red flag and a mentor will come to you.

Error or problem	Likely cause	Fix
`429` or "quota exceeded" from Gemini	Too many requests on your key	Wait 1 minute and try again. Test with short prompts. Tell your mentor if it continues.
`API key not valid` from Gemini	Wrong or missing `GEMINI_API_KEY`	Copy the key again from AI Studio, with no spaces. Restart `npm run dev`.
Model not found / no longer available	Old or wrong `GEMINI_MODEL`	Set `GEMINI_MODEL=gemini-3.8-flash` (or the model named in the error), restart, and redeploy on Vercel
`relation "..." does not exist`	Seed file not run	Run the seed SQL in the Supabase SQL Editor
"Could not find the table ... in the schema cache"	The app points to another Supabase project, or the API cache is old	Match the project id in `SUPABASE_URL` with your dashboard. Then run `notify pgrst, 'reload schema';`
`Invalid API key` from Supabase	Wrong key, or a key from another project	Use the secret (service_role) key from the same project, not the publishable or anon key
Agent replies but never calls a tool	Weak tool description or system prompt	Make the tool `description` say exactly when to use it. Name the tools in the system prompt.
Zod error on tool arguments	Gemini sent a field in the wrong format	Make the field description clearer, for example "date in YYYY-MM-DD format"
Vercel build fails	TypeScript or lint error	Run `npm run build` on your laptop first and fix every error
App works locally but not on Vercel	Env variables missing on Vercel	Add all 5 variables in Vercel project settings, then redeploy
Passcode page keeps coming back	`APP_PASSCODE` not set or browser cookies blocked	Set `APP_PASSCODE` in `.env.local` and on Vercel
`git push` denied (403)	Signed in to a different GitHub account	Sign in with the repo owner's account in VS Code, or ask the Team Lead to add you as a collaborator
`git push` rejected	A teammate pushed first	Run `git pull --rebase`, fix any conflicts, then push
Port 3000 already in use	Another app is running	Stop it, or run `npm run dev -- -p 3001`

Help order: your own teammates first, then this table, then your mentor, then the trainer.